

问题

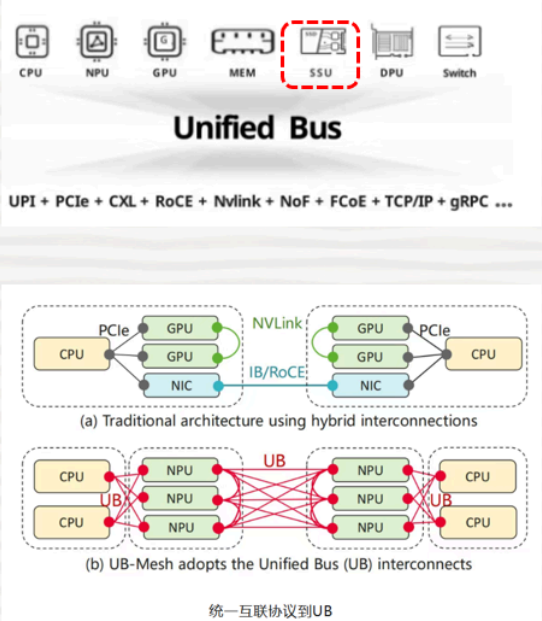


● UB

华为的 **UB (Unified Bus, 统一总线)**，中文名为“**灵衢**”，是华为为了应对 AI 大模型时代算力瓶颈而自主研发的一套**全场景互联架构**。

其设计目标是构建一个可以用于连接数万计算部件的、横向扩容的互联结构，并提供接近内存总线的高带宽能力，同时能够最小化传输时延。

让**网络像总线一样快**，让**总线能像网络一样无限扩展**！



The diagram illustrates the Unified Bus (UB) architecture. At the top, a row of icons represents various components: CPU, NPU, GPU, MEM, SSU (highlighted with a red dashed box), DPU, and Switch. Below these is the title 'Unified Bus' and a list of supported protocols: UPI + PCIe + CXL + RoCE + Nvlink + NoF + FCoE + TCP/IP + gRPC ...

Below this, two sub-diagrams are shown:

- (a) Traditional architecture using hybrid interconnections: This diagram shows two separate clusters. Each cluster has a CPU connected to a GPU and a NIC. The GPU is connected to the NIC via NVLink. The two clusters are connected to each other via IB/RoCE.
- (b) UB-Mesh adopts the Unified Bus (UB) interconnects: This diagram shows a mesh-like structure where multiple CPUs, NPUs, and GPUs are interconnected using a single Unified Bus (UB) protocol, represented by red lines.

统一互联协议到UB

好的，针对这张关于 **SSU** 和 **UB (Unified Bus / 灵衢)** 的PPT页面，我为你整理了复试老师最可能提出的问题以及相应的参考答案。

这些问题旨在考察你对项目背景、技术选型、架构设计以及行业趋势的理解深度。

● 第一部分：核心概念与背景理解 (必问基础)

？ Q1: 请简单介绍一下你项目中的 **SSU** 和 **UB** 是什么？它们之间有什么关系？

参考回答：

- **SSU (Solid State Unit)**：是我们项目中的核心智能存储单元。它不同于传统SSD，集成了计算（CPU）、存储（NAND Flash）和网络（NIC）能力，可以看作一个独立的智能节点。

- **UB (Unified Bus, 统一总线)**: 中文名“灵衢”，是华为为应对AI大模型时代算力瓶颈而自主研发的全场景互联架构。它的目标是让网络像内存总线一样快（低时延、高带宽），同时让总线能像网络一样无限扩展。
- **关系**: SSU 是构建在 UB 架构之上的一个关键组件。通过 UB 高速互联，多个 SSU 可以组成一个庞大的分布式存储池，为 AI 集群提供高效的数据供给。可以说，UB 是“高速公路网”，而 SSU 是网上的“智能仓库”。

❓ Q2: 为什么需要研发 UB 这样的新架构？传统的 PCIe + RoCE/NVLink 组合有什么不足？

参考回答：

传统架构（如PPT图a所示）存在几个主要问题：

1. **协议碎片化**: CPU-GPU用PCIe，GPU-GPU用NVLink，服务器间用RoCE/IB。多种协议并存，增加了系统复杂性和开发维护成本。
2. **性能瓶颈**: 不同协议间的转换和桥接会引入额外的延迟和开销。例如，数据从GPU显存到另一台服务器的GPU显存，可能需要经过多次拷贝和协议封装。
3. **扩展性受限**: NVLink等高速总线扩展距离和规模有限，难以支撑万卡级别的超大规模集群。

UB的优势在于它试图用一套统一的协议栈取代上述所有异构互联，实现端到端的直通，从而获得极致的性能和简化的架构，更好地适配AI大模型的训练需求。

❓ Q3: PPT中提到“让网络像总线一样快，让总线能像网络一样无限扩展”，这句话怎么理解？

参考回答：

这句话精辟地概括了UB的设计哲学：

- **“让网络像总线一样快”**: 指的是UB通过硬件卸载、零拷贝等技术，将网络通信的延迟降低到接近片内或板级总线的水平（微秒甚至纳秒级），满

足AI训练中梯度同步等对时延极度敏感的操作。

- **“让总线能像网络一样无限扩展”**：指的是UB打破了传统总线（如PCIe）在物理距离和设备数量上的限制，可以通过交换机轻松连接成千上万个计算节点（CPU/GPU/NPU/SSU），构建超大规模的算力集群。

● 第二部分：技术细节与架构设计 (深入追问)

❓ Q4: 在你的SSU项目中，RDMA技术是如何应用的？它带来了哪些具体收益？

参考回答：

我们在SSU中深度集成了RDMA（Remote Direct Memory Access）技术，主要体现在：

1. **零拷贝 (Zero Copy)**：应用层数据可以直接从用户态缓冲区传输到SSU的网卡，再发送到网络，无需经过内核缓冲区的多次拷贝，极大降低了CPU占用和内存带宽消耗。
2. **内核旁路 (Kernel Bypass)**：应用程序可以直接操作网卡寄存器发起I/O请求，避免了频繁的用户态-内核态上下文切换，显著降低了延迟。
3. **协议卸载 (Protocol Offload)**：TCP/IP或RoCE协议栈由SSU上的专用硬件处理，解放了Host CPU，使其能专注于计算任务。

收益：对于AI训练场景，这意味着更高的有效吞吐量、更低的端到端延迟，以及更少的CPU资源争抢，从而加速整个训练过程。

❓ Q5: 你提到了SSU支持盘间直连（Disk-to-Disk），这是如何实现的？相比传统方式有什么优势？

参考回答：

- **实现方式**：在传统架构中，如果要拷贝SSD A的数据到SSD B，数据流是 `SSD A -> Host Memory -> SSD B`，必须经过Host CPU和内存。而在我

们的SSU架构中，通过UB网络和UCP（控制平面）的调度，可以直接建立 **SSD A -> Network -> SSD B** 的数据通路，数据不经过Host。

- **优势：**

1. **卸载Host CPU：** Host CPU完全不参与数据搬运，可以全力运行AI训练任务。
2. **降低延迟：** 减少了数据在Host内存中的拷贝和处理环节。
3. **节省内存带宽：** 不占用宝贵的系统内存带宽，这对于内存带宽敏感的AI应用至关重要。

❓ Q6: UB-Mesh 架构（PPT图b）相比传统混合互联架构（PPT图a）有哪些具体的改进？

参考回答：

UB-Mesh 的核心改进在于**统一化和扁平化**：

1. **协议统一：** 不再需要为不同的设备对（CPU-GPU, GPU-GPU, Server-Server）维护不同的协议栈（PCIe, NVLink, RoCE等），全部统一为UB协议，简化了软硬件设计。
2. **拓扑优化：** 采用Mesh网状互联，提供了更高的冗余度和路径选择灵活性。任意两个节点间可能存在多条可达路径，有利于负载均衡和故障容错。
3. **性能提升：** 消除了协议转换网关带来的延迟和瓶颈，实现了真正的端到端高性能通信。
4. **可扩展性：** 更容易横向扩展，只需增加节点并接入UB网络即可，无需复杂的层级结构调整。

● 第三部分：工程实践与挑战 (体现动手能力)

❓ Q7: 在开发SSU的过程中，你遇到的最大技术挑战是什么？你是如何解决的？

参考回答：

(这是一个开放性问题，你可以结合自己的实际经历)

示例回答方向：

“最大的挑战之一是**保证在高并发I/O下的系统稳定性**。特别是在网络发生Linkdown等异常时，如何快速检测、优雅降级并自动恢复，而不影响上层业务。”

“我的解决方案是设计了一套健壮的状态机和错误恢复机制。具体来说：

1. 利用中断和轮询双重机制快速感知链路状态变化。
2. 一旦检测到Linkdown，立即暂停相关队列，防止数据丢失。
3. 触发预设的复位序列，尝试重新初始化物理层和数据链路层。
4. 链路恢复后，重建连接并通知上层重试未完成的I/O。

这个过程涉及到底层寄存器操作、并发控制和状态管理，非常考验对系统整体的把握能力。”

？ Q8: 如果让你来优化SSU的性能，你会从哪些方面入手？

参考回答：

我会从以下几个层面考虑优化：

1. **硬件层面**：探索使用更快的介质（如Optane）、更高带宽的SerDes、或者集成更强大的协处理器（如FPGA/ASIC）来进一步卸载计算密集型任务（如压缩/解压、加密/解密）。
2. **固件/驱动层面**：
 - 优化中断合并（Interrupt Coalescing）策略，平衡延迟和CPU开销。
 - 改进DMA引擎的效率，减少描述符处理的开销。
 - 优化缓存算法，提高热点数据的命中率。
3. **软件/协议层面**：
 - 深入研究UB协议栈，寻找可以进一步削减的握手或确认步骤。
 - 探索与上层AI框架（如PyTorch/TensorFlow）的深度协同，实现更细粒度的I/O调度和预取。

● 第四部分：跨专业与未来规划 (综合素养)

❓ Q9: 你是材料专业跨考计算机，你觉得你的背景对这个项目有什么独特的帮助吗？

参考回答：

虽然我的本科专业是材料，但我认为它带给我一些独特的优势：

1. **扎实的数理基础**：材料科学对数学和物理的要求很高，这培养了我严谨的逻辑思维能力和解决复杂问题的能力，这些能力在理解底层系统原理和设计算法时非常重要。
2. **系统工程视角**：材料研究往往需要从微观结构到宏观性能的全面考量，这种系统工程的思维方式让我在设计驱动和架构时，不仅能关注局部最优，更能考虑整体系统的稳定性和效率。
3. **强烈的学习动力**：跨专业的经历证明了我有强大的自学能力和适应新领域的决心。我对计算机系统底层充满了热情，并愿意为此付出持续的努力。

❓ Q10: 你对未来研究生阶段的研究方向有什么设想？如何与你之前的工作经验结合？

参考回答：

我希望在未来研究中，能够继续深耕“**系统+AI**”的交叉领域。

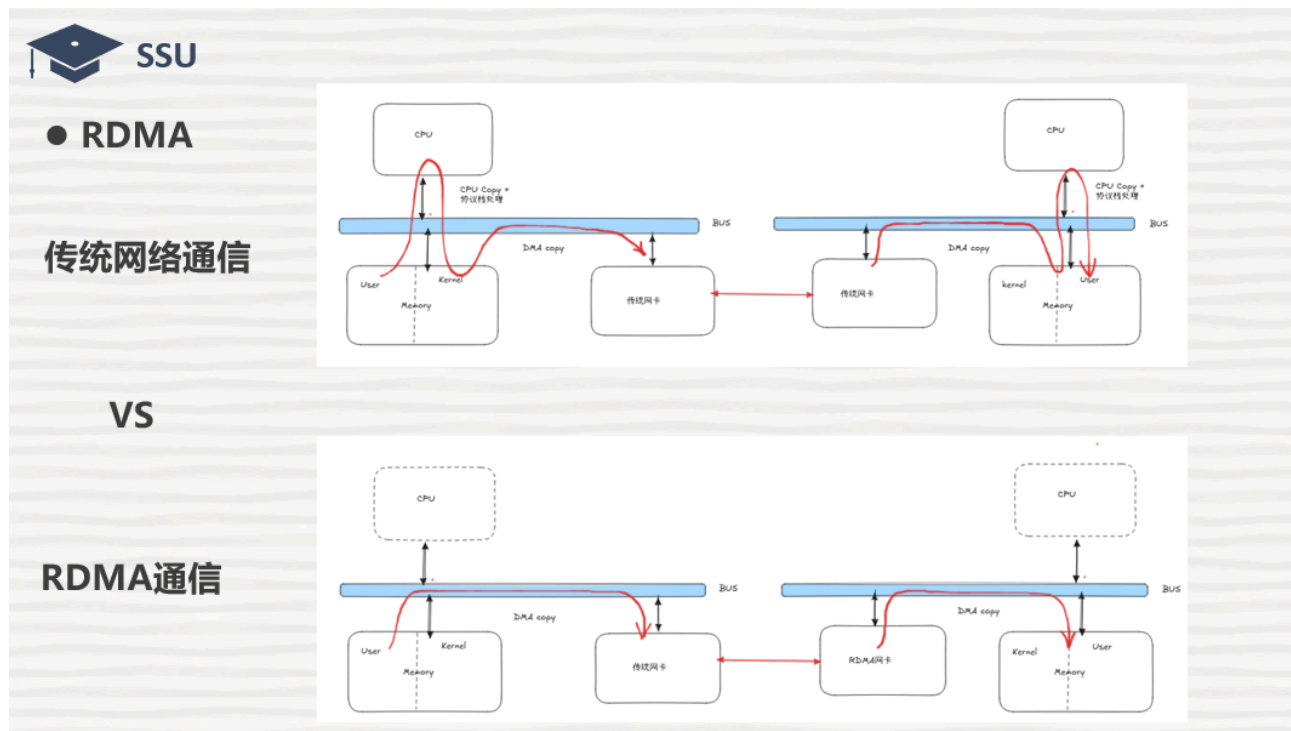
- **短期**：夯实计算机理论基础，特别是操作系统、分布式系统和机器学习原理。
- **中期**：结合我在SSU和UB方面的实践经验，研究如何进一步优化面向大模型训练的存储和网络系统。例如，探索近存计算（Processing-in-Memory）、智能数据预取、或者基于UB架构的新型一致性协议。
- **长期**：目标是成为既懂底层硬件/系统，又懂上层AI应用的复合型人才，为解决AI基础设施的瓶颈问题贡献自己的力量。

💡 复习小贴士

- **关键词记忆：**

- **UB/灵衢：**统一总线，低时延，高带宽，无限扩展，替代 PCIe/NVLink/RoCE。
- **SSU：**智能存储，CPU+Disk+NIC，RDMA，盘间直连，卸载Host。
- **RDMA：**零拷贝，内核旁路，协议卸载。
- **UB-Mesh：**统一协议，网状拓扑，高扩展，高性能。
- **画图辅助：**面试时如果能手绘出传统架构 vs UB-Mesh架构的对比图，会非常加分！
- **自信表达：**强调你的**工程实践经验**和**解决问题的能力**，这是你区别于纯理论考生的最大优势。

祝你复试顺利，展现出最好的自己！加油！💪🎓



这张图非常经典，是理解 **RDMA (Remote Direct Memory Access)** 核心价值的“金标准”示意图。它通过对比传统网络通信和 RDMA 通信的数据路径，直观地展示了性能差异的来源。

下面我为你详细解释两种通信方式的特点与实现，并整理出复试老师最可能问的问题及参考答案。

一、传统网络通信 vs. RDMA 通信：特点与实现详解

1. 传统网络通信 (Traditional Network Communication)

✓ 特点：

- **高 CPU 占用**：CPU 需要参与数据拷贝、协议栈处理、上下文切换。
- **高延迟**：多次内存拷贝 + 内核态/用户态切换 + 协议栈处理耗时。
- **低吞吐效率**：有效带宽被大量系统开销消耗。
- **复杂度高**：依赖操作系统内核网络栈，调试和优化困难。

⚙️ 实现流程（以 TCP/IP 为例）：

1. **应用层发送**：用户程序调用 `send()`，数据从 User Buffer 拷贝到 Kernel Socket Buffer。
2. **协议栈处理**：内核 TCP/IP 栈进行封装（加头部、校验、分片等），消耗 CPU。
3. **DMA 拷贝到网卡**：内核将数据通过 DMA 拷贝到网卡缓冲区。
4. **网络传输**：网卡发送数据包。
5. **接收端反向操作**：
 - 网卡 DMA 数据到 Kernel Buffer。
 - 内核协议栈解包、校验。
 - 数据从 Kernel Buffer 拷贝到 User Buffer。
6. **CPU 全程参与**：每一步几乎都需要 CPU 干预或等待。

💡 图中红色箭头清晰显示了：**两次 CPU Copy + 一次 DMA Copy + 协议栈处理**，且 CPU 在两端都深度介入。

2. RDMA 通信 (RDMA Communication)

✓ 特点：

- **零拷贝 (Zero Copy)**: 数据直接在用户内存和网卡之间搬运, 无需经过内核缓冲区。
- **内核旁路 (Kernel Bypass)**: 应用直接操作网卡寄存器, 绕过内核协议栈。
- **协议卸载 (Protocol Offload)**: TCP/RoCE/iWARP 等协议由网卡硬件完成。
- **极低延迟 & 高吞吐**: CPU 几乎不参与数据传输, 释放算力给业务。
- **适合大规模分布式系统**: 如 AI 训练、高性能计算、金融交易。

🔧 实现流程 (以 RoCEv2 为例):

1. 注册内存 (Memory Registration):

- 应用调用 `ibv_reg_mr()`, 内核锁定用户页 (Pin Pages), 生成 MR (Memory Region) 和 RKey。

2. 构建 Work Request (WR):

- 应用在用户态构造 WR (包含源/目的地址、长度、RKey 等), 写入网卡的 Send Queue。

3. 门铃通知 (Doorbell Ring):

- 应用写网卡寄存器 (Doorbell), 通知网卡有新任务。

4. 网卡自主执行:

- 网卡读取 WR, 直接使用 RKey 访问用户物理内存 (DMA Read)。
- 网卡硬件封装 RoCE 包头, 通过网络发送。

5. 接收端自动写入:

- 对端网卡收到包, 根据 QP 和 RKey 直接将数据 DMA 写入目标用户内存。
- 触发 Completion Event, 通知应用完成。

6. CPU 几乎不参与: 仅在发起和结束时轻量级介入。

💡 图中虚线框表示 CPU “空闲”, 实线箭头显示: **仅一次 DMA Copy, 无 CPU Copy, 无内核参与。**

🔍 二、复试老师可能问的问题及参考答案

❓ Q1: 请结合这张图，解释为什么 RDMA 能显著降低延迟和提升吞吐？

参考回答：

从图中可以看到，传统通信中数据需要经过 **User → Kernel → NIC → Network → NIC → Kernel → User** 的完整路径，其中包含：

- 至少 **两次 CPU 拷贝** (User↔Kernel)
- 多次 **上下文切换** (用户态↔内核态)
- 内核协议栈的 **串行处理开销**

而 RDMA 通信中，数据路径简化为 **User ↔ NIC ↔ Network ↔ NIC ↔ User**，实现了：

- **零拷贝**：数据直接从用户内存到网卡，再直达对端用户内存。
- **内核旁路**：应用直接操作网卡，避免陷入内核。
- **协议卸载**：网卡硬件处理打包/解包，CPU 不干预。

这些优化使得 RDMA 的延迟可低至微秒级，吞吐接近线速，特别适合 AI 训练、分布式存储等场景。

❓ Q2: RDMA 的“零拷贝”是如何实现的？会不会有安全风险？

参考回答：

实现方式：

1. 应用调用 **ibv_reg_mr()** 注册用户缓冲区，内核会：
 - 锁定该区域的物理页（防止 swap）
 - 记录物理地址映射
 - 生成一个唯一的 **rkey** (Remote Key)
2. 网卡在传输时，使用 **rkey + 虚拟偏移** 计算出物理地址，直接 DMA 访问。

安全机制：

- **权限控制**：每个 MR 都有访问权限（本地读/写、远程读/写），网卡硬件会检查。
- **RKey 验证**：只有持有正确 RKey 的远程节点才能访问对应内存区域。
- **地址范围限制**：MR 注册时指定了起始地址和长度，越界访问会被拒绝。

所以虽然绕过了内核，但通过硬件级的权限和地址校验，保证了安全性。

❓ Q3: 图中 RDMA 通信的 CPU 是“虚线框”，代表什么？真的完全不需要 CPU 吗？

参考回答：

虚线框表示 **CPU 在数据传输过程中基本不参与**，但并非完全不需要：

- **初始化阶段**：CPU 需要配置 QP（Queue Pair）、注册 MR、设置 CQ（Completion Queue）等。
- **启动传输**：CPU 需要写 Doorbell 寄存器触发网卡工作。
- **完成通知**：当传输完成后，网卡会更新 CQ，CPU 需要轮询或中断获取完成事件，通知应用。

但在**数据搬运的核心过程**中，CPU 是完全解放的，可以并行处理其他任务。这正是 RDMA 的价值所在 —— 让 CPU 专注于计算，而不是搬运数据。

❓ Q4: 如果我要在你的 SSU 项目中集成 RDMA，需要做哪些改造？

参考回答：

主要涉及三个层面：

1. 硬件层：

- 集成支持 RDMA 的 RNIC (RDMA NIC)，如 Mellanox ConnectX 系列。
- 确保 PCIe 带宽足够，支持 ATS/PASID 等高级特性。

2. 固件/驱动层：

- 实现 MR 注册、QP 管理、CQ 处理等 RDMA 核心功能。
- 在 UBP (数据平面) 中集成 DMA 引擎，支持直接从 NAND Flash 到 RNIC 的数据通路。

3. 软件/API 层：

- 提供兼容 libibverbs 的接口，让上层应用能方便地使用 RDMA。
- 在 FCP (前端控制平台) 中增加 RDMA 命令解析和权限校验模块。

最终目标是让 SSU 成为一个真正的“RDMA-enabled Smart Storage Node”。

? Q5: RDMA 有哪些应用场景？在你的研究中如何结合？

参考回答：

典型应用场景：

- **AI 大模型训练**：梯度同步 (AllReduce)、参数服务器通信。
- **高性能计算 (HPC)**：MPI 消息传递。
- **分布式数据库**：节点间数据复制、事务协调。
- **金融高频交易**：超低延迟订单撮合。

在我的研究中的结合点：

我希望探索 “**RDMA + 智能存储**” 的深度融合：

- 利用 RDMA 实现 **SSU 间的直连数据迁移**，无需 Host 参与。
- 结合 GDS (GPUDirect Storage)，实现 **GPU 显存 ↔ SSU ↔ 远程 GPU** 的端到端零拷贝。
- 设计基于 RDMA 的 **分布式一致性协议**，用于多 SSU 协同服务 AI 训练任务。

目标是打造一个“感知 AI 负载、主动优化数据流”的智能存储系统。

🔍 Q6: 传统网络和 RDMA 在错误处理上有何不同？比如丢包怎么办？

参考回答：

- **传统 TCP**：由内核协议栈负责重传、拥塞控制、超时检测。CPU 深度参与，逻辑复杂但成熟稳定。
- **RDMA (RoCEv2/iWARP)**：
 - **可靠连接 (RC)**：类似 TCP，网卡硬件自动重传、确认，对应用透明。
 - **不可靠连接 (UC)**：应用自己处理丢包（常用于广播/组播）。
 - **优势**：即使重传，也由网卡硬件完成，不消耗 CPU。
 - **挑战**：调试更复杂，需要专用工具（如 `perf`，`rdma-tool`）监控 QP 状态、重传计数等。

在我的项目中，我会优先选用 RC 模式保证可靠性，并通过监控 CQ 错误码来实现快速故障定位。

💡 三、记忆口诀 & 面试技巧

🎯 关键词速记：

概念	关键词
传统通信	CPU Copy, Kernel Copy, Protocol Stack, High Latency
RDMA	Zero Copy, Kernel Bypass, Protocol Offload, Low Latency
安全	Pin Pages, RKey, Permission Check, Hardware Enforcement
应用	AI Training, HPC, Distributed DB, Financial Trading

💬 面试话术模板：

“正如这张图所示，传统通信的瓶颈在于...而 RDMA 通过...解决了这些问题。在我的项目中，我们计划...”

“虽然 RDMA 看似‘绕过’了内核，但实际上通过...机制保障了安全性和可靠性。”

“我认为未来的方向是...而我的研究将聚焦于...”

✅ 总结

这张图不仅是技术对比，更是你展示**系统思维**和**工程能力**的绝佳素材。记住：

- **画图辅助**：面试时可手绘简图说明数据路径。
- **联系实际**：always tie back to your project (SSU, UB, Linkdown, Interrupt).
- **突出优势**：强调你既有理论理解，又有落地经验。

祝你复试顺利，用这张图征服评委！💪🎓

